

UNIVERSIDAD DE ANTIOQUIA
FACULTAD DE INGENIERÍA
INFORMÁTICA II

El Tormenta Escarlata

Informe de desarrollo — Proyecto Final

Vela en el Mundo Pirata

Equipo #16

Lina Vanegas Gallo
Elvia Gomez Palmar

Programación Orientada a Objetos

6 de junio de 2026

Índice

1. Introducción	3
2. Planteamiento del problema	3
3. Definición general y objetivos	3
4. Especificación de requerimientos	3
5. Metodología y planificación	4
6. Diseño y arquitectura del sistema	4
7. Desarrollo e implementación	5
8. Procedimientos de prueba	7
9. Guía de instalación y uso	7
10.Resultados y discusión	7
11.Conclusiones	8
12.Referencias	8
13.Anexos	8

1. Introducción

El Tormenta Escarlata es un videojuego de dos niveles desarrollado en C++ con Qt6 como proyecto final de Informática II. El jugador controla al Capitán Barbarroja, un marino pirata con dos características permanentes: es **Borracho** (el barco oscila lateralmente de forma involuntaria) y es **Temerario**.

El juego combina físicas matemáticas reales, programación orientada a objetos y un agente inteligente que aprende el comportamiento del jugador. Se desarrolló en Qt Creator con MinGW 64-bit sobre Windows.

2. Planteamiento del problema

El proyecto busca aplicar los conceptos de Informática II — herencia, polimorfismo, encapsulamiento y composición — en un contexto práctico y entretenido. Se eligió un videojuego porque permite integrar todos esos conceptos de forma natural: las entidades del juego forman jerarquías de clases, los niveles encapsulan su propia lógica, y el agente Kraken demuestra comportamiento autónomo con aprendizaje.

3. Definición general y objetivos

Objetivo general

Desarrollar un videojuego funcional de dos niveles en C++ con Qt6 que implemente físicas matemáticas, colisiones, sprites animados y un agente inteligente.

Objetivos específicos

- Implementar dos niveles con mecánicas distintas usando `QGraphicsScene/QGraphicsView`.
- Aplicar funciones sinusoidales y parabólicas como físicas del juego.
- Desarrollar un agente autónomo con ciclo percepción–razonamiento–acción–aprendizaje.
- Integrar sprites reales con animación por spritesheet.

4. Especificación de requerimientos

Requisitos funcionales

- El jugador controla el barco con W/A/S/D.
- El Nivel 1 tiene scroll lateral infinito con olas y viento.
- El Nivel 2 tiene scroll vertical con Kraken y barriles.
- El juego muestra HUD con salud, tiempo y puntuación.
- Al perder aparece pantalla de Game Over.
- Al completar el Nivel 2 aparece pantalla de victoria.

Requisitos no funcionales

- Framerate de aproximadamente 60 fps con `QTimer` a 16ms.
- Sprites en formato PNG con fondo transparente.
- Compatible con Qt6 y MinGW 64-bit en Windows.

Restricciones

- No se usan librerías externas fuera de Qt6.
- El proyecto debe compilar desde `Proyecto_final.pro`.

5. Metodología y planificación

Se usó un enfoque iterativo e incremental: primero se construyó el Nivel 1 funcional con `Widget`, luego se diseñó la arquitectura del Nivel 2 como clase autónoma, y finalmente se integraron ambos con transición automática.

Las fases fueron:

1. Diseño del diagrama de clases (Momento II).
2. Implementación del Nivel 1 con sprites reales.
3. Construcción del Nivel 2 con físicas y agente Kraken.
4. Integración, pruebas y corrección de errores.
5. Redacción del informe final.

6. Diseño y arquitectura del sistema

Arquitectura general

El proyecto se divide en dos capas independientes:

- **Nivel 1 — El Archipiélago de las Tortugas:** controlado por la clase `Widget`. Usa `QGraphicsScene` con scroll lateral infinito mediante dos `QGraphicsPixmapItem` alternados. Las clases `Barco`, `Ola` y `Viento` heredan de `QObject` y su respectivo `QGraphicsItem`.
- **Nivel 2 — El Mar de la Oscuridad:** controlado por `Nivel2Oscuridad`, clase autónoma que hereda de `QObject`. Gestiona su propia `QGraphicsScene` con scroll vertical, el agente Kraken, los obstáculos parabólicos y el HUD.

Clases principales

- `Widget`: controlador del Nivel 1. Gestiona el fondo, colisiones, vidas, tiempo y la transición al Nivel 2.
- `Barco`: hereda de `QObject` y `QGraphicsPixmapItem`. Controlado con W/A/S/D.
- `Ola`: hereda de `QObject` y `QGraphicsPixmapItem`. Recorta su frame del spritesheet en el constructor y se mueve con un `QTimer` propio.

- **Viento:** hereda de `QObject` y `QGraphicsRectItem`. Reduce la velocidad del barco cuando está activo.
- **Nivel20scuridad:** gestiona el Nivel 2 completo. Contiene tres `QTimer` (game loop, animación Kraken, spawn de obstáculos).

Dependencias externas

- Qt6 (Core, Gui, Widgets)
- MinGW 64-bit
- Sprites en PNG propios

7. Desarrollo e implementación

Física 1 — Oscilación por borrachera (Nivel 1)

El barco oscila lateralmente de forma sinusoidal continua, simulando el tambaleo del capitán borracho:

$$\text{offsetX}(t) = A \cdot \sin(f \cdot t)$$

donde $A = 8$ px es la amplitud y $f = 0,05$ rad/tick. El jugador debe compensar este movimiento con A y D.

```
1 float offsetX = 8.0f * static_cast<float>(qSin(tiempoBorracho));
2 float newX = qBound(limIzq, barco->x() + offsetX, limDer);
3 barco->setX(newX);
```

Física 2 — Movimiento sinusoidal del Kraken (Nivel 2)

El Kraken se desplaza verticalmente con función sinusoidal:

$$y_{\text{kraken}}(t) = y_{\text{base}} + A \cdot \sin(\omega \cdot t)$$

La amplitud A y frecuencia ω aumentan con la agresividad del agente, haciendo el movimiento progresivamente más impredecible.

Física 3 — Caída parabólica de obstáculos (Nivel 2)

Los barriles caen desde la parte superior con trayectoria parabólica:

$$x(t) = x_0 + V_x \cdot t \quad y(t) = y_0 + V_y \cdot t + \frac{1}{2} g t^2$$

donde $g = 550$ px/s² y V_x es aleatoria para simular viento lateral.

Scroll vertical (Nivel 2)

El fondo tiene 2560 px de alto. En cada tick la cámara sube:

```
1 m_scrollY += 40.0f * DT;
2 m_by = 2400.0f - m_scrollY;
3 m_view->centerOn(m_bx, m_by);
```

El barco queda fijo visualmente en la parte inferior mientras el fondo sube.

Agente Kraken

El Kraken implementa un ciclo de cuatro etapas:

1. **Percepción:** registra si el barco se mueve a izquierda o derecha comparando la posición actual con la del tick anterior.
2. **Razonamiento:** calcula probabilidad de evasión y añade ruido inversamente proporcional a la agresividad.
3. **Acción:** reposiciona su base X e Y hacia donde el jugador tiende a ir, acercándose progresivamente.
4. **Aprendizaje:** incrementa la agresividad en cada ataque, reduciendo el intervalo entre ataques y aumentando amplitud y frecuencia.

HUD siempre visible

Para que el HUD no quede tapado por el scroll, se usa `mapToScene` para obtener la esquina superior izquierda real de la cámara:

```
1 QPointF topLeft = m_view->mapToScene(0, 0);
2 m_hudSalud->setPos(topLeft.x() + 10, topLeft.y() + 10);
```

Uso de memoria dinámica

El proyecto hace uso extensivo de memoria dinámica mediante `new` para crear los objetos del juego en tiempo de ejecución. La gestión se divide en dos estrategias:

Gestión automática por Qt: todos los objetos que se crean con un padre (`QObject* parent`) son destruidos automáticamente cuando el padre se destruye. Esto aplica a timers, textos del HUD y la escena:

```
1 m_timerLoop = new QTimer(this); // this es el padre
2 m_hudSalud = new QGraphicsTextItem();
3 m_scene = new QGraphicsScene(this);
```

Gestión manual: los objetos que se crean y destruyen durante el juego requieren `delete` explícito. Los barriles inactivos se liberan en `actualizarObjetos()`:

```
1 if (obj.item->pos().y() > m_by + 500.0f) {
2     obj.activo = false;
3     m_scene->removeItem(obj.item);
4     delete obj.item;
5     obj.item = nullptr;
6 }
```

El overlay de oscuridad también se destruye y recrea en cada tick para seguir la posición del barco, liberando el anterior antes de crear el nuevo:

```
1 if (m_overlayItem) {
2     m_scene->removeItem(m_overlayItem);
3     delete m_overlayItem;
4     m_overlayItem = nullptr;
5 }
```

Esta combinación garantiza que no haya fugas de memoria: los objetos permanentes los gestiona Qt, y los objetos temporales se liberan explícitamente cuando dejan de ser necesarios.

8. Procedimientos de prueba

Las pruebas fueron manuales e iterativas. En cada sesión de desarrollo se verificó:

- Que el juego compilara sin errores ni warnings críticos.
- Que el barco se moviera correctamente con W/A/S/D.
- Que las colisiones restaran vidas/salud correctamente.
- Que el HUD mostrara los valores actualizados.
- Que la transición entre niveles funcionara.
- Que el Kraken se acercara al barco con el tiempo.

No se implementaron pruebas automatizadas dado el alcance académico del proyecto.

9. Guía de instalación y uso

Requisitos

- Qt Creator 6.x con Qt 6.8 o superior
- Compilador MinGW 64-bit
- Windows 10/11

Controles

- **W/A/S/D** o flechas: mover el barco
- El barco oscila solo por el efecto de borrachera — hay que compensarlo manualmente

10. Resultados y discusión

Funcionamiento logrado

- Nivel 1 funcional con scroll lateral, olas animadas, viento, efecto de borrachera, HUD y Game Over.
- Nivel 2 funcional con scroll vertical, fondo nocturno, Kraken animado con IA, barriles parabólicos, efecto de linterna y pantalla de victoria.
- Transición automática entre niveles al completar el Nivel 1.

Limitaciones actuales

- No hay opción de reinicio sin cerrar el programa.
- No hay sonido.

Mejoras posibles

- Agregar música y efectos de sonido con `QMediaPlayer`.
- Implementar pantalla de reinicio.
- Agregar más tipos de obstáculos (rayos, maderos).
- Implementar las corrientes marinas del Nivel 1.

11. Conclusiones

El desarrollo de *El Tormenta Escarlata* nos permitió aplicar de forma real los conceptos de Informática II:

- **Encapsulamiento:** cada clase protege su estado. `Nivel20scuridad` concentra toda la lógica del nivel sin exponer sus miembros internos.
- **Herencia múltiple:** las clases del Nivel 1 heredan simultáneamente de `QObject` y de clases gráficas de Qt.
- **Polimorfismo:** el sistema de señales y slots desacopla los eventos de fin de nivel del controlador principal.
- **Composición:** `Nivel20scuridad` agrupa internamente el Kraken, los obstáculos y el HUD en un único game loop.
- **Agente inteligente:** el Kraken demuestra que un comportamiento adaptativo puede implementarse con estructuras simples sin algoritmos complejos de IA.

El mayor aprendizaje fue enfrentar la integración de dos arquitecturas distintas — el `Widget` directo del Nivel 1 y la clase autónoma del Nivel 2 — y resolver los conflictos de compilación, inclusiones circulares y sincronización de timers que eso generó.

12. Referencias

- Qt Documentation. *Qt 6.8 Reference*. <https://doc.qt.io/qt-6/>
- Qt Documentation. *Graphics View Framework*. <https://doc.qt.io/qt-6/graphicsview.html>
- Stroustrup, B. *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- Sprites: recursos libres de uso académico.

13. Anexos

Estructura del proyecto


```
1 Proyecto-final-Info-II/  
2     Proyecto_final.pro  
3     main.cpp  
4     widget.h / widget.cpp           # Nivel 1  
5     barco.h / barco.cpp  
6     ola.h / ola.cpp  
7     viento.h / viento.cpp  
8     nivel2oscuridad.h               # Nivel 2  
9     nivel2oscuridad.cpp  
10    fondo.qrc                       # Sprites Nivel 1  
11    fondo2.qrc                      # Sprites Nivel 2  
12    imagenes/  
13        fondo_juego.png  
14        Barco.png  
15        Olas.png  
16        Fondo_nivel2.png  
17        kraken.png  
18        barril.png
```

Fragmento clave — Agente Kraken

```
1 void Nivel2Oscuridad::razonarKraken()  
2 {  
3     // Percepcion  
4     if (m_bx < m_bxAnt - 2.0f)      m_evasionesIzq++;  
5     else if (m_bx > m_bxAnt + 2.0f) m_evasionsDer++;  
6  
7     // Razonamiento  
8     float probArriba = calcularProbabilidad();  
9     float ruido = (1.0f - m_agresividad) * 0.35f;  
10    probArriba += ruido * (aleatorio() - 0.5f);  
11  
12    // Accion  
13    m_kxBase = m_bx + 100.0f + aleatorio(200) - 100.0f;  
14    m_kyBase = (probArriba > 0.5f) ? m_by-100.0f : m_by+100.0f;  
15  
16    // Aprendizaje  
17    m_agresividad = qMin(1.0f, m_agresividad + 0.035f);  
18    m_intervaloAtaque = BASE * (1.0f - m_agresividad * 0.5f);  
19 }
```